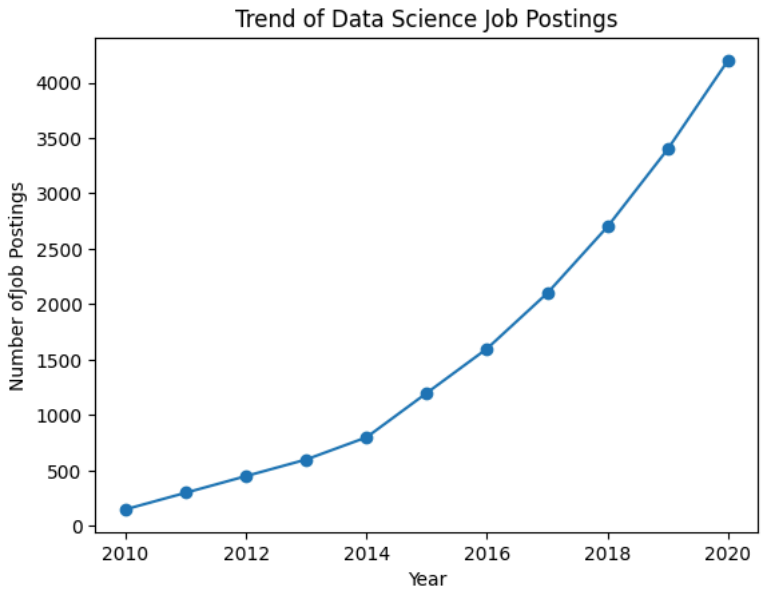
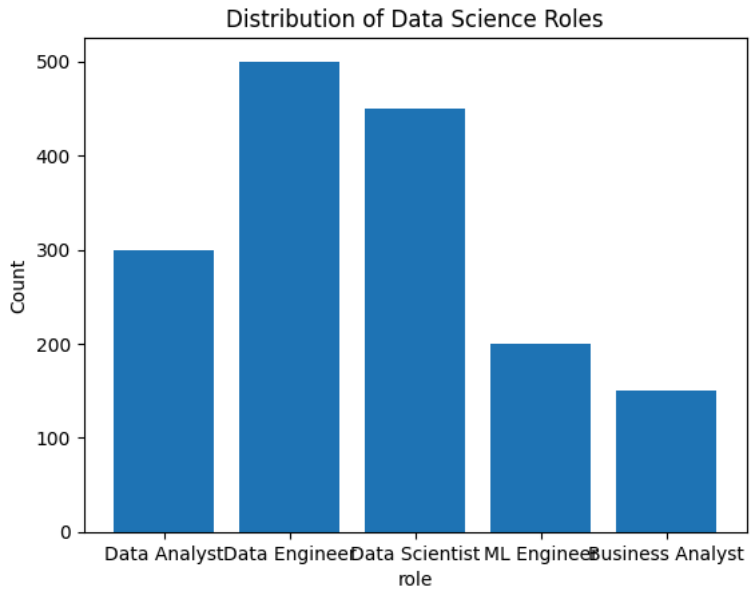
```
#1a
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()
```




```
#1b
roles=['Data Analyst','Data Engineer','Data Scientist','ML Engineer','Business Analyst']
counts=[300,500,450,200,150]
plt.bar(roles,counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('role')
plt.ylabel('Count')
plt.show()
```




```
#1c
#
structured_data = pd.DataFrame({
'ID': [1, 2, 3],
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35]
})
```

```
print("Structured Data:\n", structured_data)
#
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image,or a video file."
print("\nUnstructured Data:\n", unstructured_data)
#
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

```
Structured Data:
    ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
 This is an example of unstructured data. It can be a piece of text, an image,or a video file.

Semi-structured Data:
 {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

```
#1d
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqu0ROUhH5XGhxg4tuQ_HH43BvD3TulFpD7qo-GZuV3buxtoNNmKYg0_WhgKo0g58-1g4KgAstH2-yqbe34bbtQ0PpUASn4P73jQ
Decrypted Data: b'Rajalakshmi Engineering College.'
```

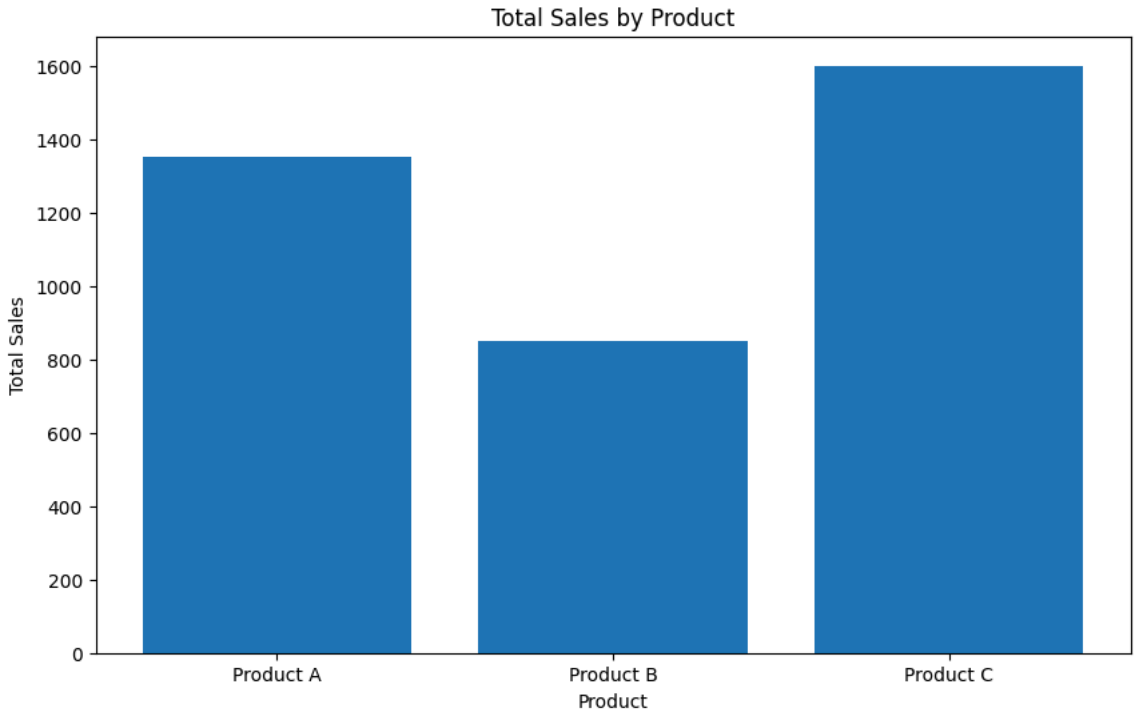
```
#2
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
file_path='sales_data.csv'
df = pd.read_csv(file_path)
print(df.head())
print(df.isnull().sum())
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print(df.describe())
product_summary = df.groupby('Product').agg({
'Sales': 'sum',
'Quantity': 'sum'
}).reset_index()
print(product_summary)

11

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc='sum', fill_value=0)
print(pivot_table)
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)
import seaborn as sns
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

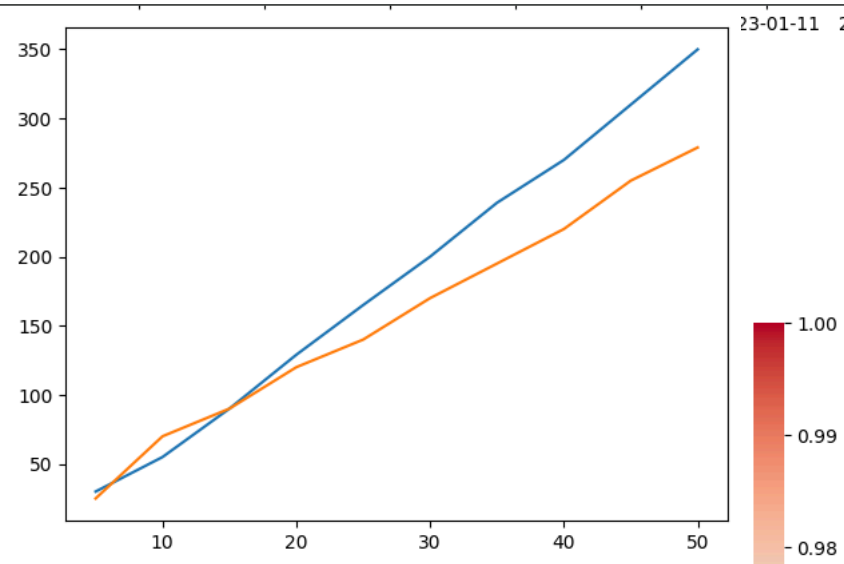
```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```



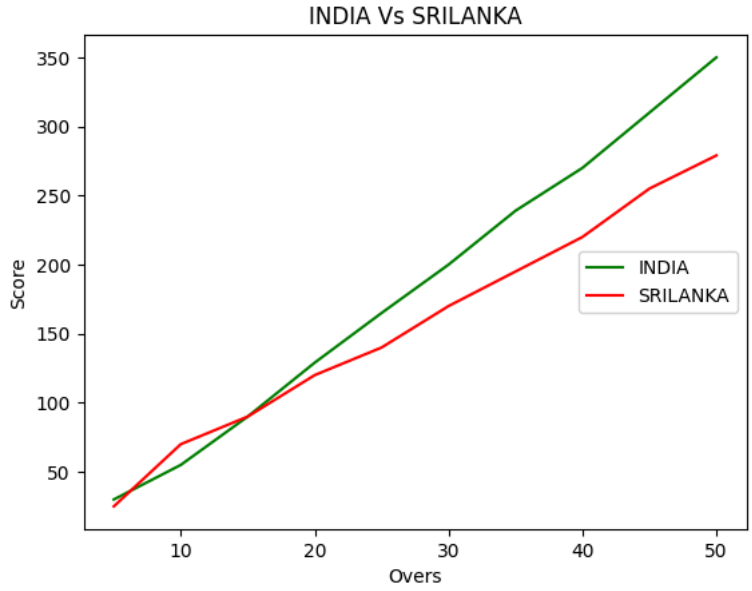
SalesOver Time

```
#3a
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

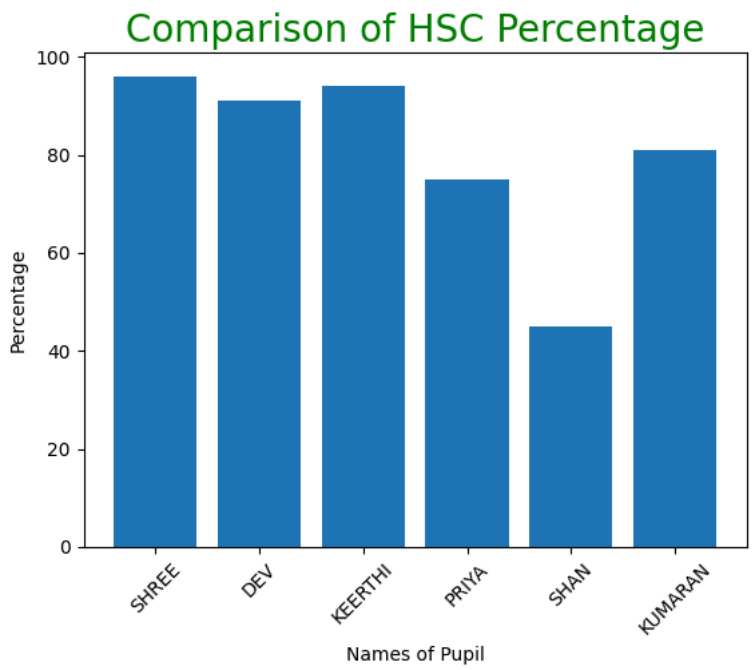




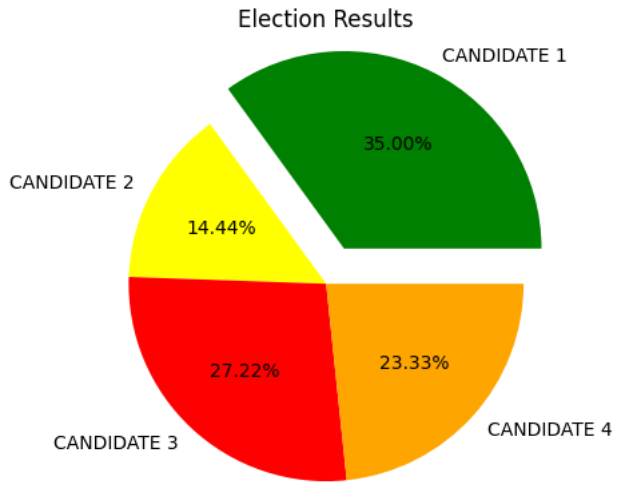

```
/tmp/ipykernel_1444/179130383.py:12: UserWarning: No artists with labels found to put in legend.  Note that artists whose la
  cricket.legend()
<matplotlib.legend.Legend at 0x7ed5eda2e850>
```




```
#3b
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```




```
#3c
import matplotlib.pyplot as election
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```




```
#4
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
    wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str([item for item in zip(wlist, wordfreq)]))
```

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

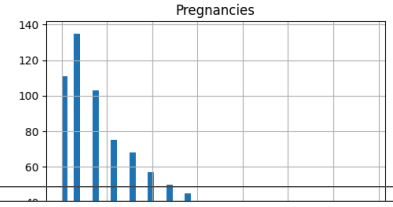
```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1
```

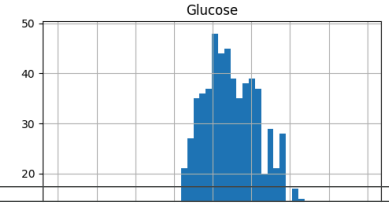
```
#5
import pandas as pd
db=pd.read_csv("diabetic_data.csv")
print(db.head())
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50,figsize=(20,15))
plt.show()
sns.pairplot(db)
plt.show()
```

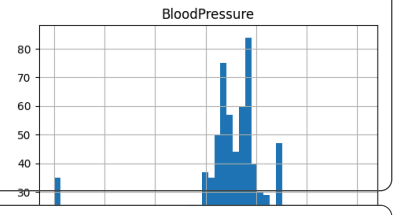
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```








```
#6
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

data = pd.read_csv(
    'website_traffic.csv',
    parse_dates=['Date'],
    index_col='Date'
)

decomposition = seasonal_decompose(
```

```
    data['Page_Views'],
    model='additive'
)

decomposition.plot()
plt.show()

data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()

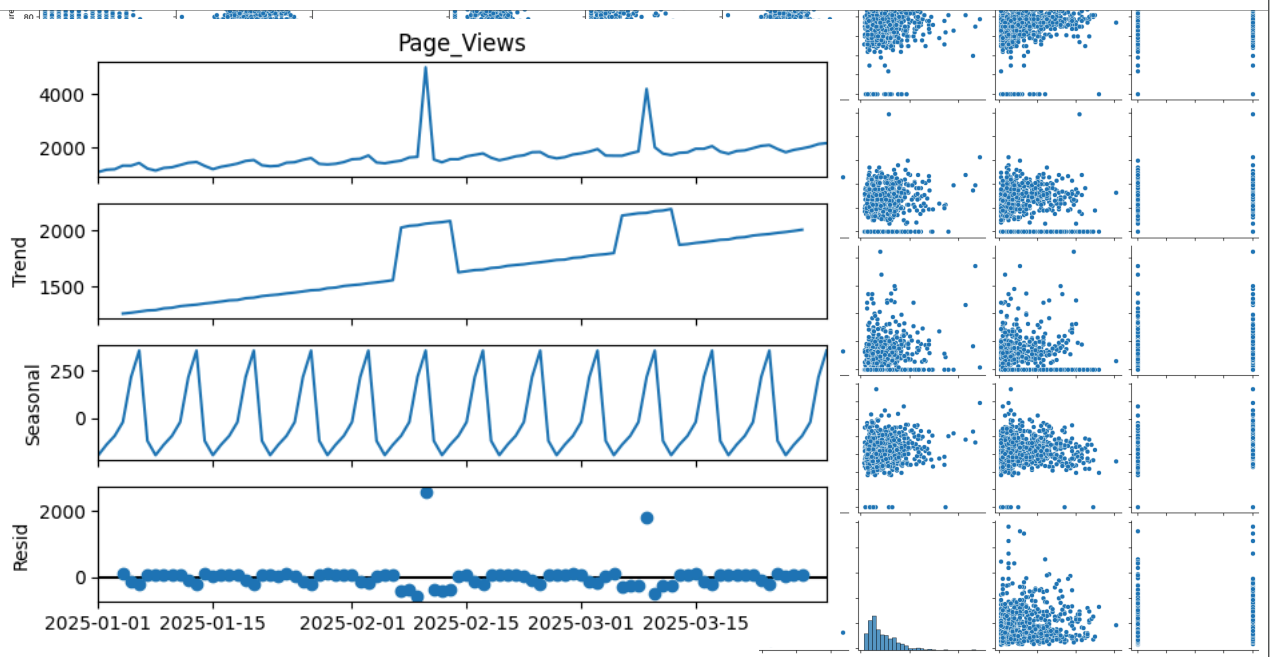
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

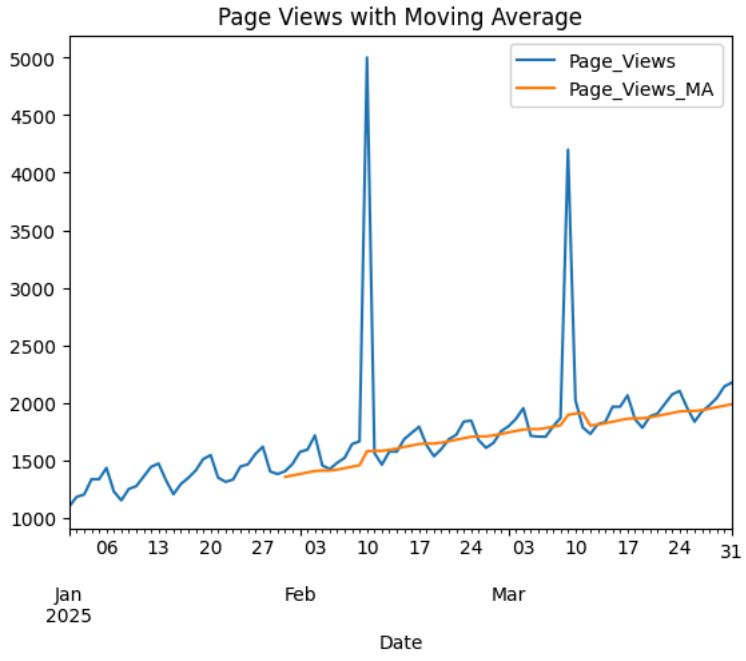
data['Page_Views_Z'] = (
    data['Page_Views'] - data['Page_Views'].mean()
) / data['Page_Views'].std()

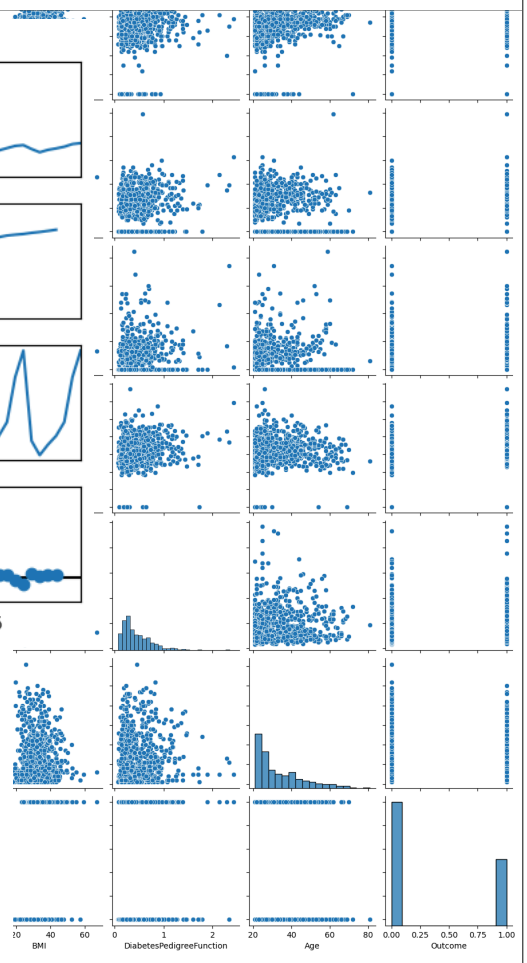
anomalies = data[data['Page_Views_Z'].abs() > 3]

plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(
    anomalies.index,
    anomalies['Page_Views'],
    color='red',
    label='Anomalies'
)

plt.legend()
plt.show()
```







```
#7
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

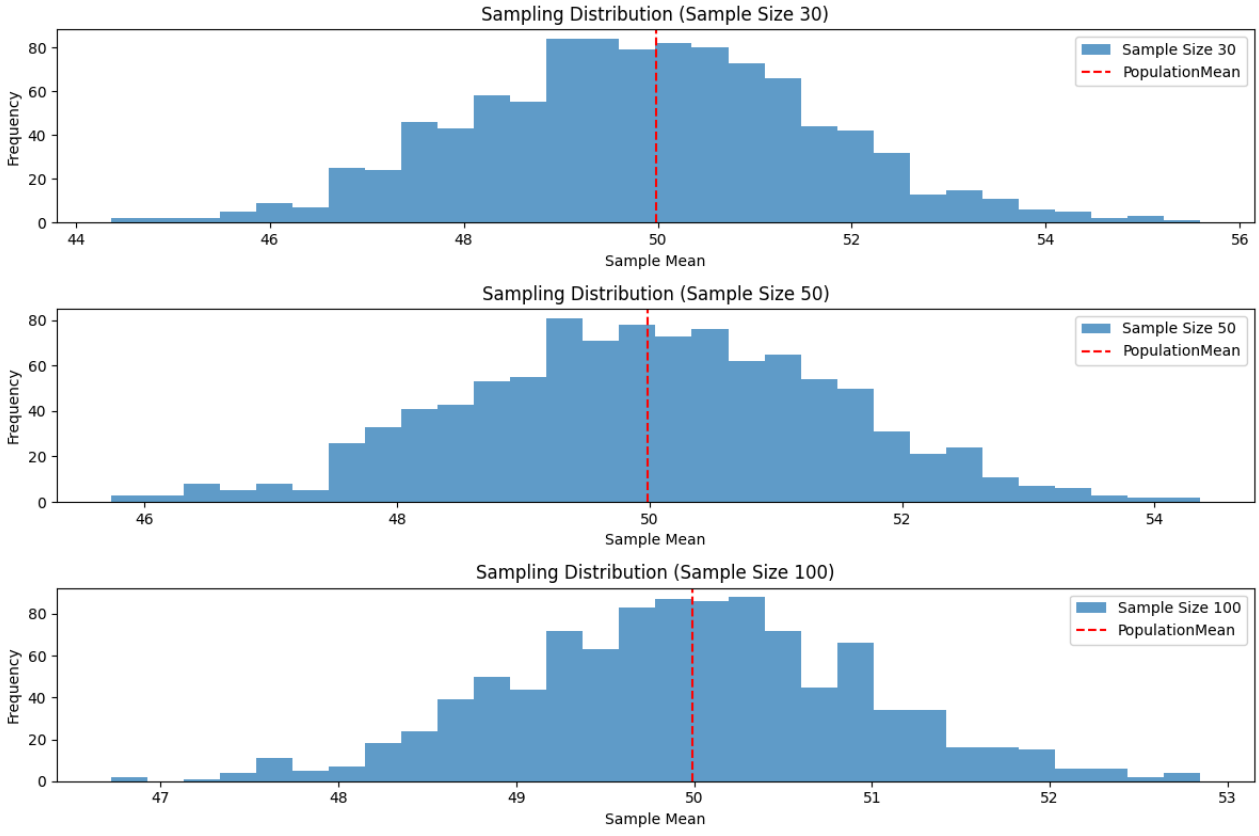
sample_sizes = [30, 50, 100]
num_samples = 1000

sample_means = {}

for size in sample_sizes:
  sample_means[size] = []
  for _ in range(num_samples):
    sample = np.random.choice(population, size=size, replace=False)
    sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
  plt.subplot(len(sample_sizes), 1, i+1)
  plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
  plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5, label='PopulationMean')
  plt.title(f'Sampling Distribution (Sample Size {size})')
  plt.xlabel('Sample Mean')
  plt.ylabel('Frequency')
  plt.legend()

plt.tight_layout()
plt.show()
```




```
#8
import numpy as np
import scipy.stats as stats
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,151, 150, 149, 152, 151,
148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])
population_mean = 150
```

```
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
  print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")
else:
  print("Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.")
```

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

```
#9
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('iris')

print("--- Dataset Overview ---")
print(data.head())
print("\n--- Data Info ---")
data.info()
print("\n--- Summary Statistics ---")
print(data.describe())
print("\n--- Species Count ---")
print(data.value_counts('species'))

plt.figure(figsize=(6, 4))
sns.countplot(x='species', data=data)
plt.title('Distribution of Iris Species')
plt.xlabel('Species')
plt.ylabel('Count')
plt.show()
dummies = pd.get_dummies(data.species)
FinalDataset = pd.concat([dummies, data.iloc[:, [0, 1, 2, 3]]], axis=1)
print("\n--- One-Hot Encoded Dataset Preview ---")
print(FinalDataset.head())

plt.figure(figsize=(7, 5))
sns.scatterplot(x='sepal_length', y='sepal_width', hue='species', data=data)
plt.title('Sepal Length vs Sepal Width')
plt.show()

plt.figure(figsize=(7, 5))
sns.scatterplot(x='petal_length', y='petal_width', hue='species', data=data)
plt.title('Petal Length vs Petal Width')
plt.show()
sns.pairplot(data, hue='species', height=3)
plt.suptitle('Pairwise Feature Relationships', y=1.02)
plt.show()
g = sns.FacetGrid(data, hue='species', height=5)
g.map(sns.histplot, 'sepal_width').add_legend()
plt.title('Distribution of Sepal Width by Species')
plt.show();

sns.FacetGrid(data, hue='species', height=5).map(sns.histplot, 'petal_width').add_legend();
plt.title('Distribution of Petal Width by Species')
plt.show();

sns.FacetGrid(data, hue='species', height=5).map(sns.histplot, 'sepal_length').add_legend();
plt.title('Distribution of Sepal Length by Species')
plt.show();

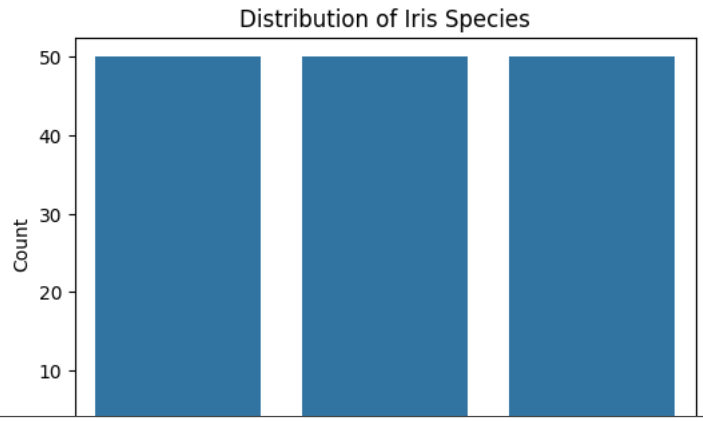
sns.FacetGrid(data, hue='species', height=5).map(sns.histplot, 'sepal_width').add_legend();
plt.title('Distribution of Sepal Width by Species')
plt.show();
```

```
--- Dataset Overview ---
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

--- Data Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

--- Summary Statistics ---
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000

--- Species Count ---
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```




```
#10
import numpy as np

array = np.random.randint(1, 100, 9)
print("Original Array:")
print(array)

print("\nSquare root of array elements:")
print(np.sqrt(array))

print("\nArray dimensions:", array.ndim)

new_array = array.reshape(3, 3)
print("\nReshaped Array (3x3):")
print(new_array)

print("\nReshaped array dimensions:", new_array.ndim)

print("\nFlattened array (ravel):")
print(new_array.ravel())

newm = new_array.reshape(3, 3)
print("\nMatrix 'newm':")
print(newm)

print("\nSlice [2, 1:3]:", newm[2, 1:3])
print("Slice [1:2, 1:3]:\n", newm[1:2, 1:3])
print("Slice [0:3, 0:0]:\n", new_array[0:3, 0:0])
print("Slice [0:2, 0:1]:\n", new_array[0:2, 0:1])
```

```
print("Slice [0:3, 0:1]:\n", new_array[0:3, 0:1])
print("Slice [1:3]:\n", new_array[1:3])
```

```
Original Array:
[42 80  7 56 89 26 24  9 94]

Square root of array elements:
[6.4807407  8.94427191 2.64575131 7.48331477 9.43398113 5.09901951
 4.89897949 3.         9.69535971]

Array dimensions: 1

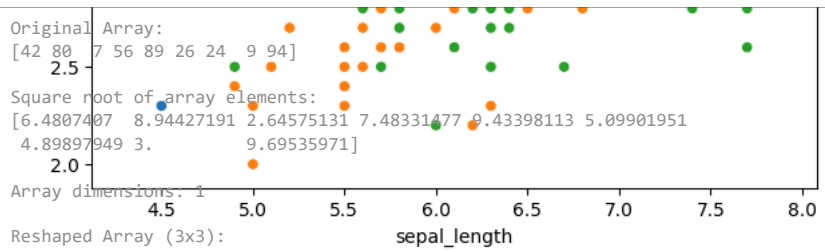
Reshaped Array (3x3):
[[42 80  7]
 [56 89 26]
 [24  9 94]]

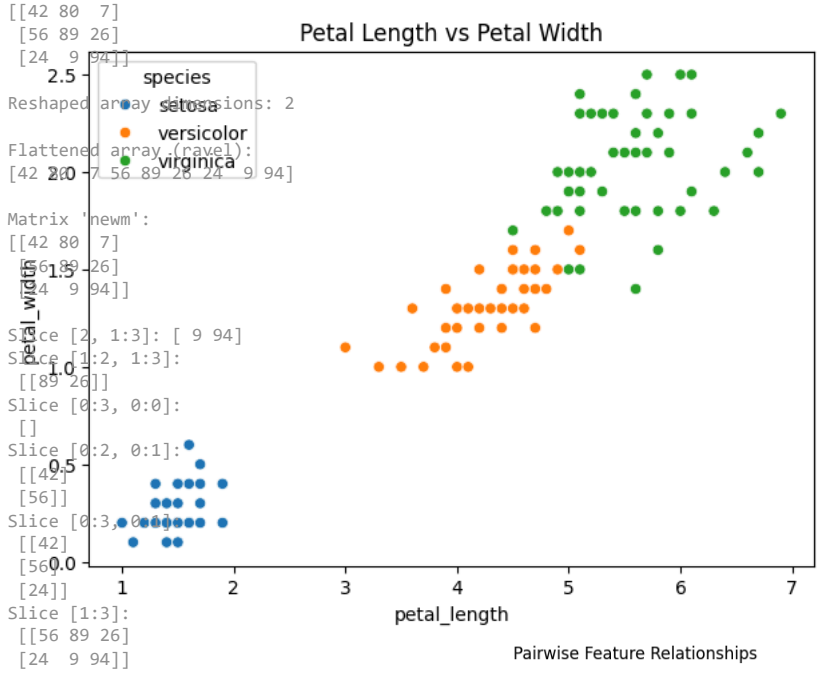
Reshaped array dimensions: 2

Flattened array (ravel):
[42 80  7 56 89 26 24  9 94]

Matrix 'newm':
[[42 80  7]
 [56 89 26]
 [24  9 94]]

Slice [2, 1:3]: [ 9 94]
Slice [1:2, 1:3]:
 [[89 26]]
Slice [0:3, 0:0]:
 []
Slice [0:2, 0:1]:
 [[42]
 [56]]
Slice [0:3, 0:1]:
 [[42]
 [56]
 [24]]
Slice [1:3]:
 [[56 89 26]
 [24  9 94]]
```





Pairwise Feature Relationships

```
#11
import numpy as np
import pandas as pd

# Renamed 'list' to 'emp_data' to avoid shadowing python built-in list type
emp_data = [[1,'Smith',50000],[2,'Jones',60000]]

df = pd.DataFrame(emp_data)
print(df)

df.columns = ['Empd','Name','Salary']
print(df)

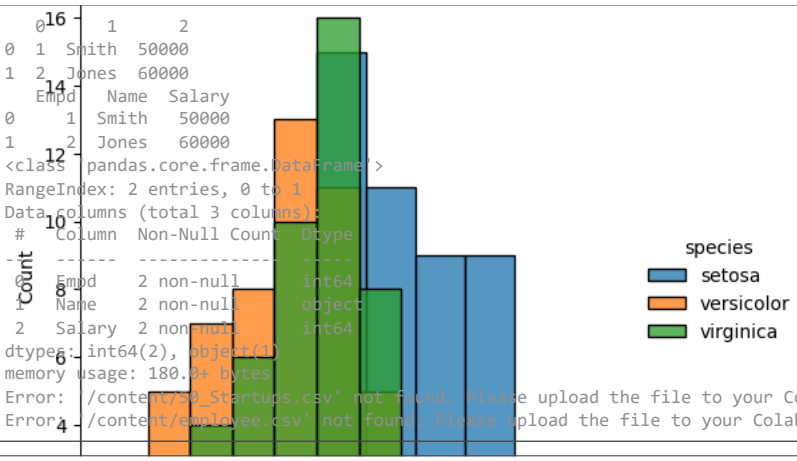
df.info()
try:
    df = pd.read_csv("/content/50_Startups.csv")
    df.info()
    print(df.head())
    print(df.tail())
except FileNotFoundError:
    print("Error: '/content/50_Startups.csv' not found. Please upload the file to your Colab workspace.")

try:
    df = pd.read_csv("/content/employee.csv")
    print(df.head())
    print(df.tail())
    df.info()

    print(df.Salary)
    print(type(df.Salary))
    print(df.Salary.mean())
    print(df.Salary.median())
    print(df.Salary.mode())
    print(df.Salary.var())
    print(df.Salary.std())
    print(df.describe())
    print(df.describe(include='all'))
    empCol = df.columns
    print(empCol)
    emparray = df.values
    print(emparray)
    employee_DF = pd.DataFrame(emparray, columns=empCol)
```

```
    print(employee_DF)
except FileNotFoundError:
    print("Error: '/content/employee.csv' not found. Please upload the file to your Colab workspace.")
```

```
   0      1      2
0  1  Smith  50000
1  2  Jones  60000
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
Error: '/content/50_Startups.csv' not found. Please upload the file to your Colab workspace.
Error: '/content/employee.csv' not found. Please upload the file to your Colab workspace.
```




```
#12
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

array = np.random.randint(1, 100, 16)
print(array)

print(array.mean())
print(np.percentile(array, 25))
print(np.percentile(array, 50))
print(np.percentile(array, 75))
print(np.percentile(array, 100))

def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
print(lr, ur)

sns.displot(array)

sns.distplot(array)

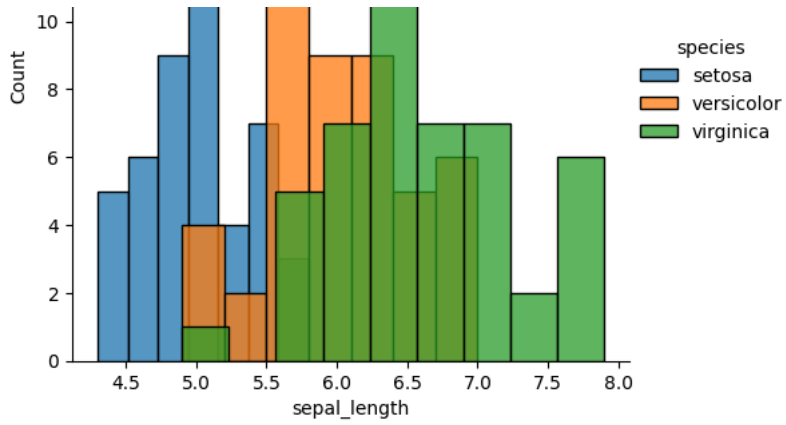
new_array = array[(array > lr) & (array < ur)]
print(new_array)

sns.displot(new_array)

lr1, ur1 = outDetection(new_array)
print(lr1, ur1)

final_array = new_array[
    (new_array > lr1) & (new_array < ur1)
]
print(final_array)

sns.distplot(final_array)
```



```
[30  5 48 94  4 77 89 46 55 35 27 28 42 21 83 50]
45.875
27.75
44.0
60.5
94.0
-21.375 109.625
/tmp/ipykernel_867/2077053667.py:28: UserWarning:

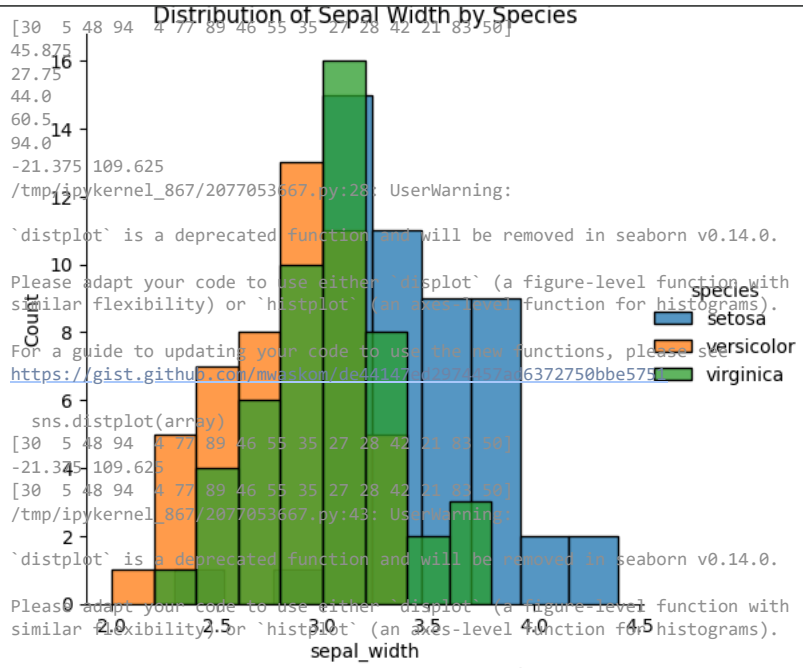
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

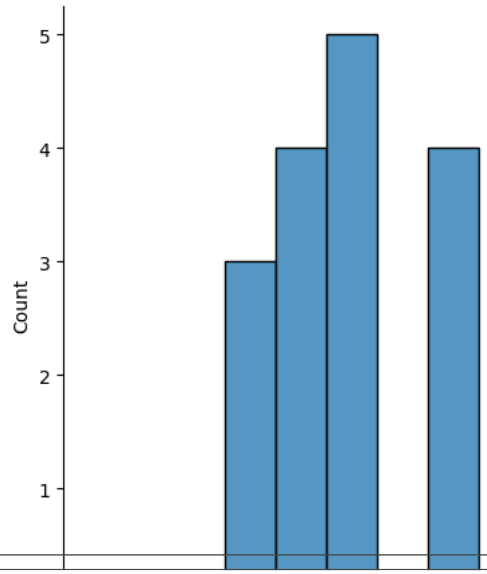
Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).
For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(array)
[30  5 48 94  4 77 89 46 55 35 27 28 42 21 83 50]
-21.375 109.625
[30  5 48 94  4 77 89 46 55 35 27 28 42 21 83 50]
/tmp/ipykernel_867/2077053667.py:43: UserWarning:
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).
For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(final_array)
<Axes: ylabel='Count'>
```






```
#13
import numpy as np
import pandas as pd

df = pd.read_csv("Hotel_Dataset.csv")

print(df)

print(df.duplicated())

df.info()

df.drop_duplicates(inplace=True)

print(df)

print(len(df))
index = np.arange(len(df))

df.set_index(index,inplace=True)

print(index)

df.drop(['Age_Group.1'],axis=1,inplace=True)

print(df)

df.CustomerID.loc[df.CustomerID<0] = np.nan
```

```
df.Bill.loc[df.Bill<0] = np.nan

df.EstimatedSalary.loc[df.EstimatedSalary<0] = np.nan

print(df)

df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)] = np.nan

print(df)

print(df.Age_Group.unique())

print(df.Hotel.unique())

df.Hotel.replace(['Ibys'],'Ibis',inplace=True)

print(df.FoodPreference.unique())

df.FoodPreference.replace(
    ['Vegetarian','veg'],
    'Veg',
    inplace=True
)

df.FoodPreference.replace(
    ['non-Veg'],
    'Non-Veg',
    inplace=True
)

df.EstimatedSalary.fillna(
    round(df.EstimatedSalary.mean()),
    inplace=True
)

df.NoOfPax.fillna(
    round(df.NoOfPax.median()),
    inplace=True
)

df['Rating(1-5)'].fillna(
    round(df['Rating(1-5)'].median()),
    inplace=True
)

df.Bill.fillna(
    round(df.Bill.mean()),
    inplace=True
)

print(df)
```

```
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
9            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg  6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
9         3          -99999       25-30
10        4           87777       30-35
0     False
1     False
2     False
3     False
4     False
5     False
6     False
```

```
7     False
8     False
9      True
10    False
dtype: bool
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree         Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
```

```
#14
import numpy as np
import pandas as pd

df = pd.read_csv('/content/pre-process_datasample.csv')

print(df)

print(df.head())

df.Country.fillna(df.Country.mode()[0],inplace=True)

features = df.iloc[:,:-1].values

label = df.iloc[:,-1].values

from sklearn.impute import SimpleImputer

age = SimpleImputer(
    strategy="mean",
    missing_values=np.nan
)

Salary = SimpleImputer(
    strategy="mean",
    missing_values=np.nan
)

age.fit(features[:,[1]])

Salary.fit(features[:,[2]])

features[:,[1]] = age.transform(features[:,[1]])

features[:,[2]] = Salary.transform(features[:,[2]])

print(features)

from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)

Country = oh.fit_transform(features[:,[0]])

print(Country)

final_set = np.concatenate(
    (Country,features[:,[1,2]]),
    axis=1
)

print(final_set)

from sklearn.preprocessing import StandardScaler

sc = StandardScaler()

sc.fit(final_set)
```

```
feat_standard_scaler = sc.transform(final_set)

print(feat_standard_scaler)

from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0,1))

mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)

print(feat_minmax_scaler)
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]]
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
```

```
#15
import numpy as np
import pandas as pd

df = pd.read_csv("/content/pre-process_datasample.csv")

print(df)

df.info()

print(df.Country.mode())

print(df.Country.mode()[0])
```

```
print(type(df.Country.mode()))

df.Country.fillna(
    df.Country.mode()[0],
    inplace=True
)

df.Age.fillna(
    df.Age.median(),
    inplace=True
)

df.Salary.fillna(
    round(df.Salary.mean()),
    inplace=True
)

print(df)

print(pd.get_dummies(df.Country))

updated_dataset = pd.concat(
    [
        pd.get_dummies(df.Country),
        df.iloc[:,[1,2,3]]
    ],
    axis=1
)

print(updated_dataset)

df.info()

updated_dataset.Purchased.replace(
    ['No','Yes'],
    [0,1],
    inplace=True
)

print(updated_dataset)
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
0    France
Name: Country, dtype: object
France
<class 'pandas.core.series.Series'>
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
```

```
6   False    False   True
7    True    False  False
8   False     True  False
9    True    False  False
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
```

```
#16
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

tips=sns.load_dataset('tips')

tips.head()

sns.displot(tips.total_bill,kde=True)
plt.show()

sns.displot(tips.total_bill,kde=False)
plt.show()

sns.jointplot(x=tips.tip,y=tips.total_bill)
plt.show()

sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
plt.show()

sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
plt.show()

sns.pairplot(tips)
plt.show()

tips.time.value_counts()

sns.pairplot(tips,hue='time')
plt.show()

sns.pairplot(tips,hue='day')
plt.show()

sns.heatmap(tips.corr(numeric_only=True),annot=True)
plt.show()

sns.boxplot(tips.total_bill)
plt.show()

sns.boxplot(tips.tip)
plt.show()

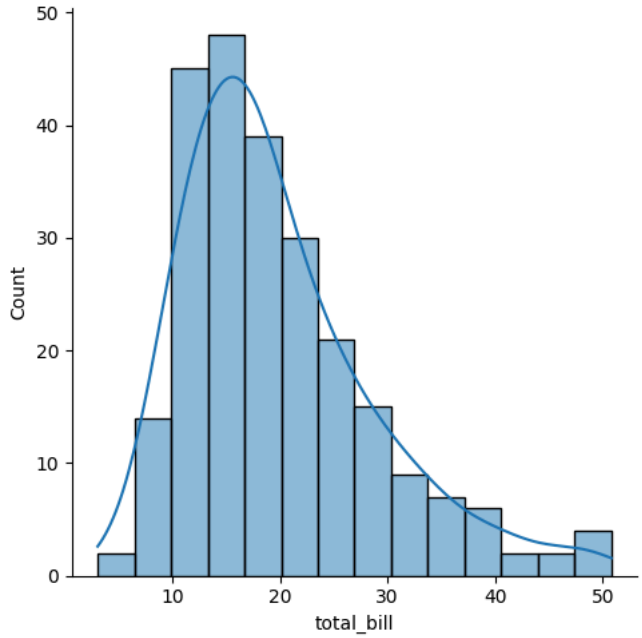
sns.countplot(tips.day)
plt.show()

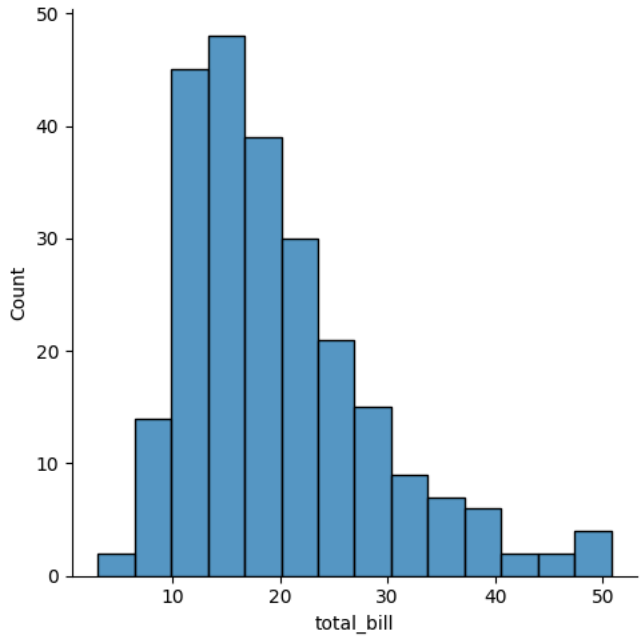
sns.countplot(tips.sex)
plt.show()

tips.sex.value_counts().plot(kind='pie')
plt.show()

tips.sex.value_counts().plot(kind='bar')
plt.show()

sns.countplot(tips[tips.time=='Dinner']['day'])
plt.show()
```






```
#17
import numpy as np
import pandas as pd

df=pd.read_csv('Salary_data.csv')

df

df.info()

df.dropna(inplace=True)

df.info()

df.describe()

features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values

from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=42)

from sklearn.linear_model import LinearRegression

model=LinearRegression()

model.fit(x_train,y_train)
```

```
model.score(x_train,y_train)

model.score(x_test,y_test)

model.coef_

model.intercept_

import pickle

pickle.dump(model,open('SalaryPred.model','wb'))

model=pickle.load(open('SalaryPred.model','rb'))

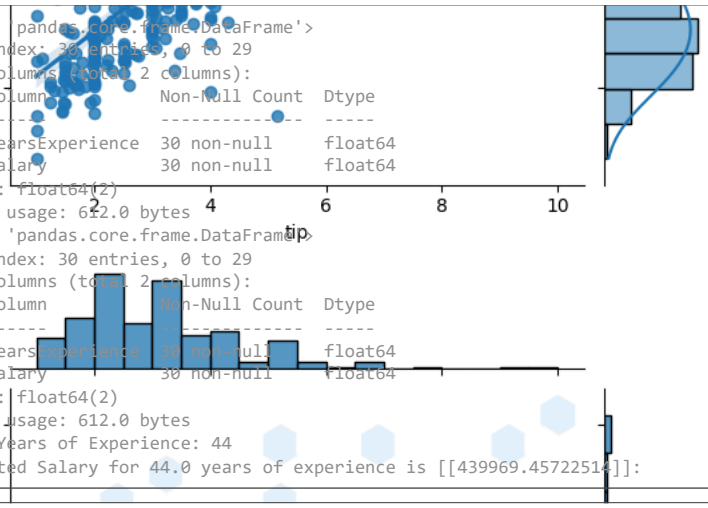
yr_of_exp=float(input("Enter Years of Experience: "))

yr_of_exp_NP=np.array([[yr_of_exp]])

Salary=model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}: ".format(yr_of_exp,Salary))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     float64
dtypes: float64(2)
memory usage: 612.0 bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     float64
dtypes: float64(2)
memory usage: 612.0 bytes
Enter Years of Experience: 44
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```




```
#18
import numpy as np
import pandas as pd

df=pd.read_csv('Iris.csv')

df.info()

print(df.variety.value_counts())

print(df.head())

features=df.iloc[:,:-1].values

label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=0.2,random_state=42)

model_KNN=KNeighborsClassifier(n_neighbors=5)

model_KNN.fit(xtrain,ytrain)

print(model_KNN.score(xtrain,ytrain))

print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix

print(confusion_matrix(label,model_KNN.predict(features)))

from sklearn.metrics import classification_report

print(classification_report(label,model_KNN.predict(features)))
```

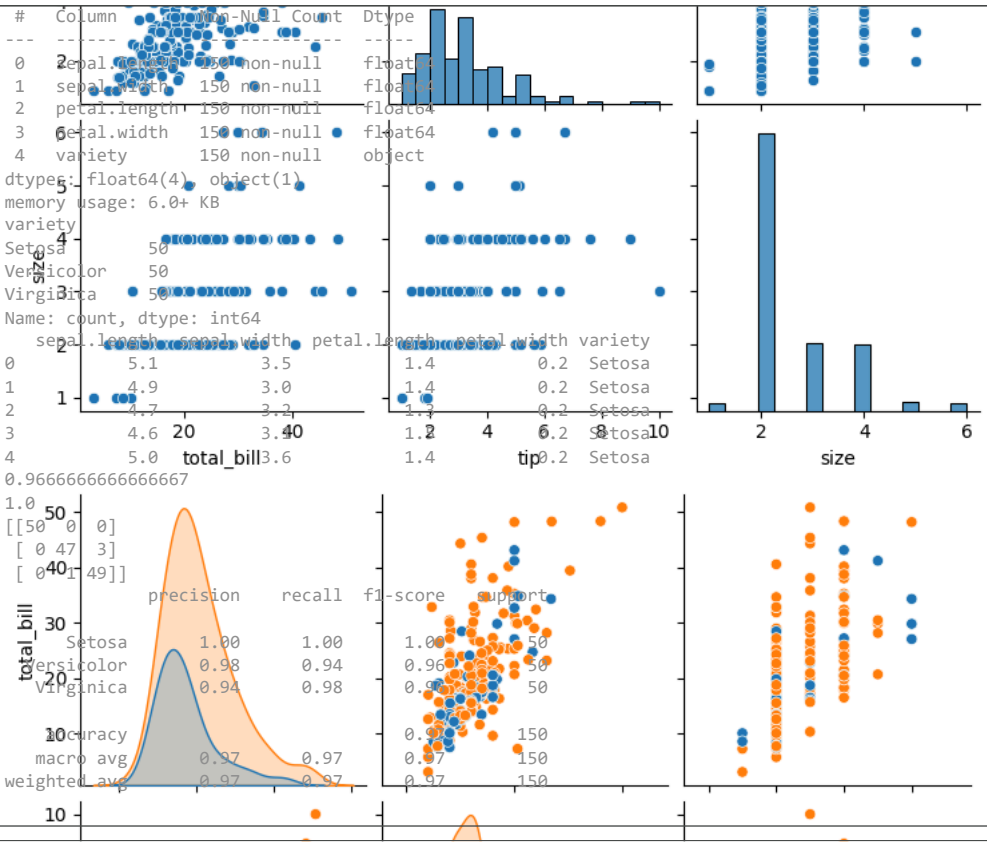
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
```



```
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
variety
Setosa       50
Versicolor   50
Virginica    50
Name: count, dtype: int64
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
0.9666666666666667
1.0
[[50  0  0]
 [ 0 47  3]
 [ 0  1 49]]
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```




```
#19
import numpy as np
import pandas as pd

df=pd.read_csv('Social_Network_Ads.csv')

df

df.head()

features=df.iloc[:,[2,3]].values

label=df.iloc[:,4].values

print(features)

print(label)

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=37)

finalModel=LogisticRegression()

finalModel.fit(x_train,y_train)

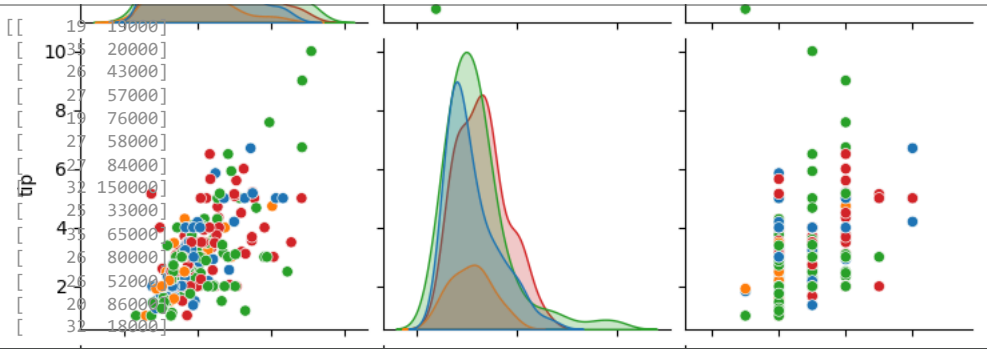
print(finalModel.score(x_train,y_train))

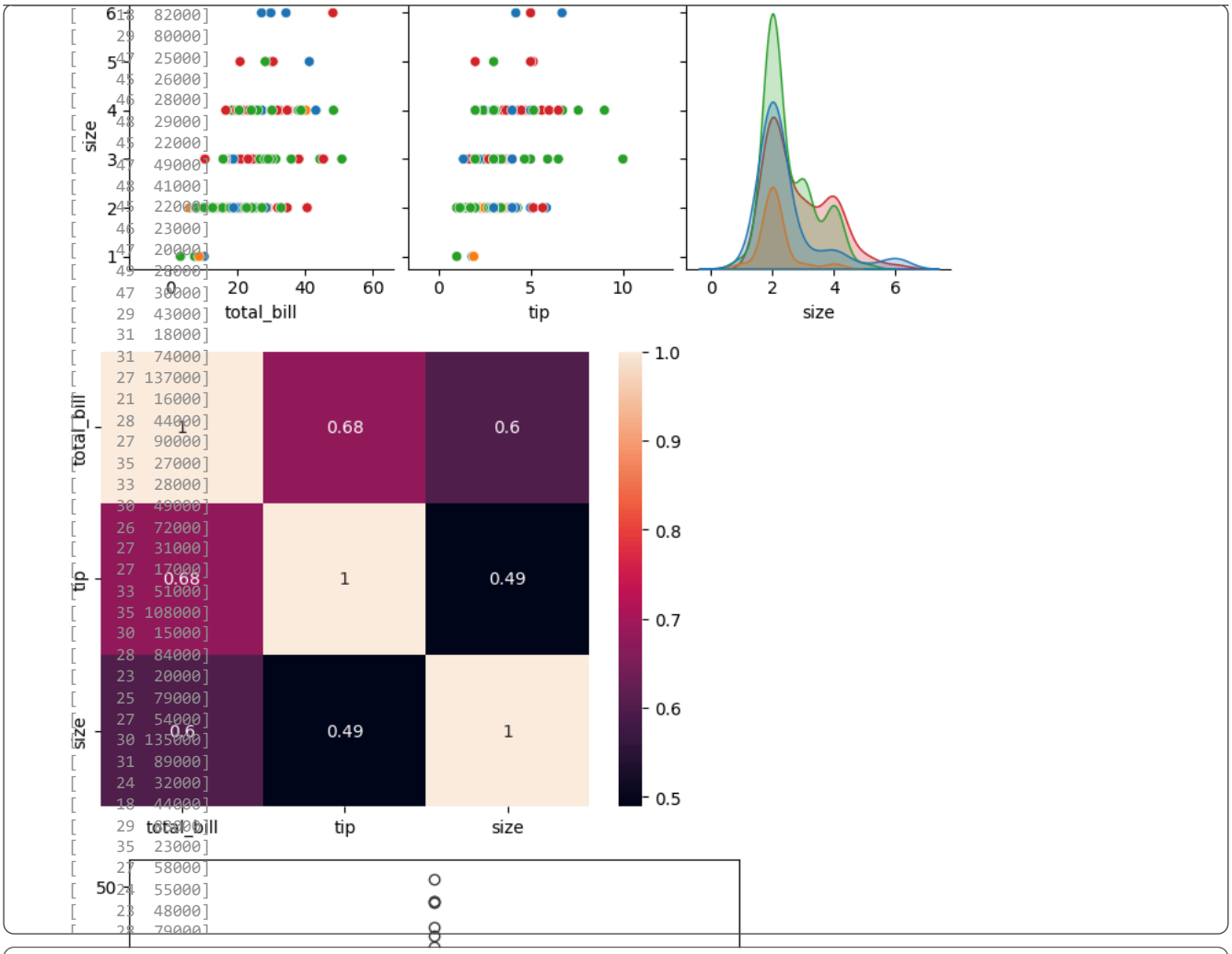
print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report

print(classification_report(label,finalModel.predict(features)))
```

```
[[    19  19000]
 [    35  20000]
 [    26  43000]
 [    27  57000]
 [    19  76000]
 [    27  58000]
 [    27  84000]
 [    32 150000]
 [    25  33000]
 [    35  65000]
 [    26  80000]
 [    26  52000]
 [    20  86000]
 [    32  18000]
```






```
#20
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

df=pd.read_csv('Mall_Customers.csv')

df.info()

print(df.head())

sns.pairplot(df)

plt.show()

features=df.iloc[:,[3,4]].values

from sklearn.cluster import KMeans

model=KMeans(n_clusters=5)

model.fit(features)

Final=df.iloc[:,[3,4]]

Final['label']=model.predict(features)

print(Final.head())

sns.set_style("whitegrid")

sns.FacetGrid(Final,hue="label",height=8).map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)").add_legend()

plt.show()

features_el=df.iloc[:,[2,3,4]].values

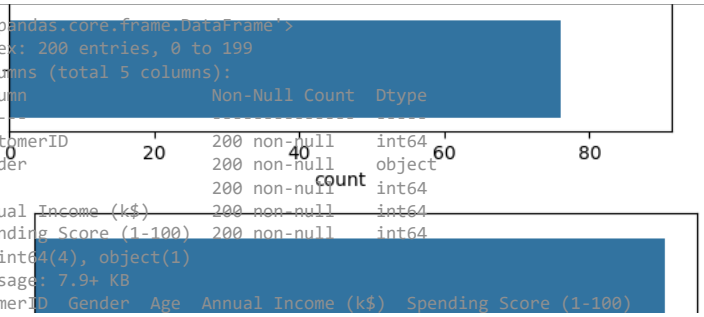
from sklearn.cluster import KMeans
```

```
wcss=[]

for i in range(1,10):
    model=KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1,10),wcss)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```




```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
```